

NetworkPeer Complete Build Dossier

Generated: 2026-09-04

Scope

NetworkPeer is a privacy-first marketplace for verified field work. The repository contains the TypeScript API, the web platform, native Android and iOS clients, shared API contracts, AWS Terraform infrastructure, CI workflows, tests, deployment runbooks, and generated documentation.

This dossier is the end-to-end operational guide. The source repository remains the authoritative complete codebase; this PDF records how to build, test, demo, deploy, audit, and troubleshoot it.

Repository layout

Area	Location	Purpose
API	`NetworkPeer-main/`	TypeScript service, migrations, unit and E2E tests
Web	`NetworkPeer-platform-main/`	Vite frontend for client, worker, and admin surfaces
Mobile	`apps/android/`, `apps/ios/`	Native Android and iOS applications
Contracts	`packages/api-contracts/`	Shared API schemas and generated contract types
Infrastructure	`infra/terraform/`	AWS networking, ECS, data, security, and observability
Documentation	`docs/`	Markdown runbooks and technical references
PDF output	`docs/pdfs/`	One flat folder containing generated PDFs

Linux Android demo, now

Prerequisites

Install Android Studio, JDK 17+, Android SDK platform 36, build-tools 35.0.0, and either an emulator or a USB-debuggable Android device. Enable virtualization for an emulator where required by the host.

Configure the app

```
```bash
cd /home/int_s.aditya/Documents/NetworkPeer-main/apps/android
cp networkpeer.development.local.properties.example
networkpeer.development.local.properties
${EDITOR:-vi} networkpeer.development.local.properties
```
```

Set `API\_BASE\_URL` to a reachable API URL ending in `/api/v1/`. Add a test Stripe publishable key only when funding is part of the demo. Do not add secrets, signing files, `local.properties`, or Firebase files to Git.

Start the API and web app

In separate terminals:

```
```bash
cd /home/int_s.aditya/Documents/NetworkPeer-main/NetworkPeer-main
npm install
npm run dev
```
```

```
```bash
cd /home/int_s.aditya/Documents/NetworkPeer-main/NetworkPeer-platform-main
```

```
npm install
npm run dev -- --host 0.0.0.0
```
```

For an Android emulator, use `10.0.2.2` instead of `localhost` in the API URL. For a physical phone, use the Linux host LAN IP and allow the API port through the local firewall.

Build, install, and launch

```
```bash
cd /home/int_s.aditya/Documents/NetworkPeer-main/apps/android
./gradlew :app:testDevelopmentDebugUnitTest
./gradlew :app:installDevelopmentDebug
adb shell monkey -p com.networkpeer.mobile.dev 1
adb logcat | grep -i networkpeer
```
```

The demo path is: OTP sign-in, choose client or worker role, create or inspect a job, review checklist/evidence state, and verify account-scoped notifications. Without configured Firebase, the core app remains usable but push registration is disabled. Without a real API, the app intentionally shows configuration/error state rather than fabricating data.

Validation matrix

```
```bash
cd /home/int_s.aditya/Documents/NetworkPeer-main/NetworkPeer-main
npm run typecheck
npm run lint
npm test
npm run test:e2e
npm run build
```
```

```
```bash
cd /home/int_s.aditya/Documents/NetworkPeer-main/NetworkPeer-platform-main
npm run lint
npm run build
```
```

```
```bash
cd /home/int_s.aditya/Documents/NetworkPeer-main/apps/android
./gradlew :app:testDevelopmentDebugUnitTest
./gradlew :app:assembleProductionRelease
```
```

```
```bash
cd /home/int_s.aditya/Documents/NetworkPeer-main/infra/terraform
terraform fmt -check -recursive
terraform init -backend=false -input=false
terraform validate
```
```

Run `npm audit --audit-level=high` in each JavaScript application before release. Review every finding rather than suppressing advisories blindly.

Security and data boundaries

The API uses Helmet, production HSTS, explicit CORS, body limits, Redis-backed rate limiting, authenticated route registration, and sanitized errors. Evidence uses API-issued short-lived presigned targets; clients do not receive AWS credentials and do not connect directly to PostgreSQL or Redis. Mobile tokens use Android Keystore-backed encrypted storage, and account-scoped

synchronization prevents stale sessions from writing into a new account.

Before production sign-off, verify endpoint-specific rate limits, object-level authorization, upload magic-byte and malware scanning, certificate pinning policy, alert delivery, SBOM generation, Terraform state protection, and IAM least privilege against the current deployment. The security audit report should be updated with evidence for each item.

Live surfaces and deployment

The currently reachable web deployment is ``https://network-peer-web.vercel.app/``. It renders the NetworkPeers landing page and links to ``/auth``, ``/client``, ``/worker``, and ``/admin``. The public GitHub repository is ``https://github.com/rudraaxl/NetworkPeer``; its ``main`` branch has the same major top-level directories as this workspace. This checkout has no ``.git`` directory, so commit equality and pushing cannot be performed from it.

After obtaining a real Git clone, synchronize safely:

```
```bash
git fetch origin
git status --short
git diff --stat origin/main...HEAD
git diff --stat origin/main
```
```

Review the diff, then commit and push only after confirming no local user changes are being overwritten. GitHub Actions should then validate the pushed commit.

PDF generation

From the workspace root:

```
```bash
python3 scripts/build_docs_pdf.py
find docs/pdfs -maxdepth 1 -type f -name '*.pdf' -print | sort
```
```

The builder converts Markdown under ``docs/`` into a single flat ``docs/pdfs/`` folder, including this dossier and legacy documentation. It preserves manually placed non-generated files and removes only generated PDF/HTML/CSS artifacts before rebuilding.

Known limitations and release gates

The dossier does not embed every source file into a PDF because the repository itself is the complete codebase and is the correct versioned source of truth. A release is not complete until API, web, Android, infrastructure, security, and live smoke checks above have passed in the target environment. Production secrets and cloud credentials must be supplied through the deployment system, never committed to this workspace.